

PYTHON COLLECTIONS

What is Python Collections?

Python Collections is a built-in module that provides alternatives to the basic collection types (list, tuple, set, dict). These specialized data structures optimize memory and performance for specific use cases.

Basic Collection Types in Python

List (list) – Ordered, mutable, allows duplicates

Tuple (tuple) – Ordered, immutable, allows duplicates

Set (set) – Unordered, mutable, no duplicates

Dictionary (dict) – Unordered, mutable, key-value pairs

Advanced Collections (from collections module)

The collections module provides additional data structures:

Counter – Counts occurrences of elements

deque – Double-ended queue for fast appends and pops

defaultdict – Dictionary with default values for missing keys

OrderedDict – Dictionary that remembers insertion order (before Python 3.7)

namedtuple – Tuple with named fields

ChainMap – Groups multiple dictionaries together

UserDict, UserList, UserString – Customizable versions of built-in types

List

In Python, we have lots of data structures, but the List is among the most basic and important. The List indexing starts from index zero and goes up to (length of the List - 1). Many operations are possible in the List like adding, multiplying, slicing, membership and comparison etc. In Python, Lists are used to

store the sequence of different data types. Python lists are mutable(changeable), which means we can change their elements after they've been formed. Lists are one of four built-in Python data structures for storing data collections; the other three are Tuple, Set, and Dictionary, all of which have different properties and applications.

Introduction

Python offers six types of built-in sequences to store data, one of which is a List. The List is like a dynamic array, along with the linear sequence, and also provides you with several built-in functions to perform various operations on data stored in it.

We will see what operations can be performed on lists later in this article. Lists in Python can contain duplicate elements as each element is identified by a unique number/ position known as an index.

We can store any number of items/elements in a list, and they can be homogeneous (all items of the same data type) or heterogeneous (items of different data types in a list).

The data stored in the List is ordered. That is, it will be stored in the way we input it. Therefore, Lists are ordered sequences. List in Python also offers various simple and shorthand ways like List comprehensions. We will also see about List comprehension later in this article.

Characteristics of a Python List

The various characteristics of a list are:

Ordered: Lists maintain the order in which the data is inserted.

Mutable: In list element(s) are changeable. It means that we can modify the items stored within the list.

Heterogenous: Lists can store elements of various data types.

Dynamic: List can expand or shrink automatically to accommodate the items accordingly.

Duplicate Elements: Lists allow us to store duplicate data.

Creating Lists in Python

A list is created by placing the items/ elements separated by a comma (,) between the square brackets ([]). Let's see how we can create Lists in Python in different ways.

Creation of a List in Python

Creating an empty List

```
empty_List = []
```

Creating a List of Integers

```
integer_List = [26, 12, 97, 8]
```

Creating a List of floating point numbers

```
float_List = [5.8, 12.0, 9.7, 10.8]
```

Creating a List of Strings

```
string_List = ["Interviewbit", "Preparation", "India"]
```

Creating a List containing items of different data types

```
List = [1, "Interviewbit", 9.5, 'D']
```

Creating a List containing duplicate items

```
duplicate_List = [1, 2, 1, 1, 3, 3, 5, 8, 8]
```

We can also create a list storing several other lists as its items. It is known as a nested list. The nested list is created by placing the various lists separated by comma (,) within the square brackets ([]). Let us see below how nested lists are created in Python.

Creating a nested List

```
nested_List = [[6, 2, 8], [1, "Interviewbit", 3.5], "preparation"]
```

Accessing Values/Elements in List in Python

Elements stored in the lists are associated with a unique integer number known as an index. The first element is indexed as 0, and the second is 1, and so on. So, a list containing six elements will have an

index from 0 to 5. For accessing the elements in the List, the index is mentioned within the index operator ([]) preceded by the list's name.

Another way we can access elements/values from the list in python is using a negative index. The negative index starts at -1 for the last element, -2 for the last second element, and so on.

elements in python list

```
My_List = [3, 4, 6, 10, 8]

# Accessing values stored in lists using positive index
print("Values accessed using positive Index.")
print(My_List[2])
print(My_List[4])

# Accessing values stored in lists using negative index
print("Values accessed using negative Index.")
print(My_List[-1])
print(My_List[-5])
```

OUTPUT:

Values accessed using positive Index.

6

8

Values accessed using negative Index.

8

3

We also have nested Lists in python. For nested Lists, we first access the index at which the inner list is stored/indexed in the outer list using the index operator. Then we access the element from the inner list using the index operator again. It is known as nested indexing. It is like we access elements in a 2-

dimensional array. Unlike in this, we have many elements in the inner list, but arrays only have a single value at that index.

```
# Nested List
```

```
nested_List = [[6, 2, 8], [1, 3.5, "Interviewbit"], "preparation"]
```

```
print(nested_List[0][1])
```

```
print(nested_List[1][2])
```

OUTPUT:

2

Interviewbit

Updating List in Python

Lists are mutable, meaning that their existing values stored at the indexes can be changed. To change or update the list elements, the assignment operator (=) is used.

Let's see the code below that shows how it's actually done.

```
My_List = [3, 4, 6, 10, 8, 5]
```

```
print(My_List)
```

```
# Changing value to index 3.
```

```
My_List[3] = 7
```

```
print(My_List)
```

```
# Changing value to index -1, i.e the last value.
```

```
My_List[-1] = 15
```

```
print(My_List)
```

OUTPUT:

```
[3, 4, 6, 10, 8, 5]
```

```
[3, 4, 6, 7, 8, 5]
```

```
[3, 4, 6, 7, 8, 15]
```

Add/Change list Elements in Python

Lists are mutable, unlike strings, so we can add elements to the lists. We use the `append()` method to add a single element at the end of the list and the `extend()` method to add multiple elements at the end of the list. We also have an `insert()` method to insert an element at the specified index position. The `insert()` method takes two arguments, the first is the index at which the element is to be inserted, and the second is the value/ element to be inserted.

```
vowels = ['a', 'e']
```

```
print(vowels)
```

```
# Appending an element in list
```

```
vowels.append('i')
```

```
print(vowels)
```

```
# Extending an element in list
```

```
vowels.extend(['o', 'u'])
```

```
print(vowels)
```

```
# Inserting an element at a specified index
```

```
Integers = [1, 5, 6, 9]
```

```
Integers.insert(1,7)
```

```
print(Integers)
```

OUTPUT:

```
['a', 'e']
```

```
['a', 'e', 'i']
```

```
['a', 'e', 'i', 'o', 'u']
```

```
[1, 7, 5, 6, 9]
```

Delete/Remove Lists Elements in Python

We can delete one or multiple elements from the list in python using the del keyword. We can even delete the whole list using this keyword. If we delete a list and then try to access it, the interpreter will generate an error.

Now let's see an example code below, how the del keyword is used.

```
# Deleting/Removing items from the List.  
my_List = [1, 2, 3, 4, 5, 6, 7, 8]  
print(my_List)  
  
# Deleting/Removing a single item from the list  
del my_List[3]  
print(my_List)  
  
# Deleting the entire List  
del my_List  
  
# The below line will generate an error as the List doesn't exist now.  
print(my_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

```
[1, 2, 3, 5, 6, 7, 8]
```

NameError: name 'my_List' is not defined

```
print(my_List)
```

Line 10 in <module> (Solution.py)

We have three more methods `remove()`, `pop()`, and `clear()` for this purpose.

The `remove()` method takes the item as an argument that must be deleted or removed. If the item isn't present within the list, a `ValueError` is raised.

The `pop()` method takes the index as an argument and deletes and returns the deleted item. If the index isn't given, `pop()` deletes and returns the list's last element.

The `clear()` method in python makes the list empty.

Deleting/Removing items from List using various methods

```
my_List = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(my_List)
```

Deleting/Removing using `remove()` method

```
my_List.remove(4)
```

```
print(my_List)
```

Deleting/Removing using `pop()` method

```
print(my_List.pop(1))
```

```
print(my_List)
```

`pop()` method with no arguments

```
print(my_List.pop())
```

```
print(my_List)
```

`clear()` method

```
my_List.clear()
```

```
print(my_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[1, 2, 3, 5, 6, 7, 8, 9]
```

```
2
```

```
[1, 3, 5, 6, 7, 8, 9]
```


9

```
[1, 3, 5, 6, 7, 8]
```

```
[]
```

List Operations in Python

Let's see the various operations we can perform on the list in Python.

Concatenation: One list may be concatenated with itself or another list using '+' operator.

```
List1 = [1, 2, 3, 4]
```

```
List2 = [5, 6, 7, 8]
```

```
concat_List = List1 + List2
```

```
print(concat_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Repeating the elements of the list: We can use the '*' operator to repeat the list elements as many times as we want.

```
my_List = [1, 2, 3, 4]
```

```
print(my_List*2)
```

OUTPUT:

```
[1, 2, 3, 4, 1, 2, 3, 4]
```

Membership Check of an element in a list: We can check whether a specific element is a part/ member of a list or not.

```
my_List = [1, 2, 3, 4, 5, 6]
```

```
print(3 in my_List)
print(10 in my_List)
print(10 not in my_List)
```

OUTPUT:

True

False

True

Iterating through the list: We can iterate through each item within the list by employing a for a loop.

```
my_List = [1, 2, 3, 4, 5]
```

```
for n in my_List:
```

```
    print(n)
```

OUTPUT:

1

2

3

4

5

Finding the length of List in python: len() method is used to get the length of the list.

```
my_List = [1, 2, 3, 4, 5]
```

```
print(len(my_List))
```

OUTPUT:

Slicing a List in Python

In Python, we can access the elements stored in a range of indexes using the slicing operator (:). The index put on the left side of the slicing operator is inclusive, and that mentioned on the right side is excluded.

Slice itself means part of something. So when we want to access elements stored at some part or range in a list we use slicing.

```
my_List = ['i', 'n', 't', 'v', 'i', 'e', 'w', 'b', 'i', 't']  
# Accessing elements from index beginning to 5th index  
print(my_List[:6])  
# Accessing elements from index 3 to 6  
print(my_List[3:7])  
# Accessing elements from index 5 to end  
print(my_List[5:])  
# Accessing elements from beginning to end  
print(my_List[:])  
# Accessing elements from beginning to 4th index  
print(my_List[:-6])
```

OUTPUT:

```
['i', 'n', 't', 'v', 'i', 'e']  
['v', 'i', 'e', 'w']  
['e', 'w', 'b', 'i', 't']  
['i', 'n', 't', 'v', 'i', 'e', 'w', 'b', 'i', 't']  
['i', 'n', 't', 'v']
```

We can also use slicing to change, remove/delete multiple elements from the list. Let's see some code examples and understand them.

```
List = [1, 2, 3, 4, 5, 6, 7, 8]

print(List)

# Changing elements from index 2nd to 5th
List[2:5] = [1, 1, 1]

print(List)

# Deletion multiple items from the List
del List[2:6]

print(List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 1, 1, 1, 6, 7, 8]
[1, 2, 7, 8]
```

List Methods in Python

In python, the method is like a function that performs some task but can only be called on objects. Methods are related to objects and may change the object's state. Let's see some python list methods.

S. No.	Method	Description of method
1	append()	Add an item to the ending of the list.
2	extend()	Add items of one list to another list.
3	insert()	Adds an item at the desired index position.
4	remove()	Remove the primary item from the list with the required value.
5	pop()	Remove the item from the required index within the list, and return it.
6	clear()	Remove all items from the list.

- 7 `index()` Returns 0-based index of the first matched item. Raises `ValueError` if an item isn't found within the list.
- 8 `count()` Returns the frequency of an element appearing in the list.
- 9 `sort()` Sorts the objects of the list. It uses a compare function if passed as an argument.
- 10 `reverse()` Reverse the order of the list items in place.
- 11 `copy()` Return a shallow copy of the list i.e. two lists share the identical elements via reference.

Built-in List Methods/Functions in Python

Functions are a group of instructions that typically operate on sequences or objects in python. Python provides plenty of built-in functions for lists. Here is a list of some built-in functions associated with lists in python.

S. No.	Function	Description
1	<code>len(my_List)</code>	Calculates and returns the length or size of the list
2	<code>max(my_List)</code>	Returns the item with maximum value within the list
3	<code>min(my_List)</code>	Returns the item with minimum value within the list
4	<code>list(iterable)</code>	Return the list containing the elements of the iterable.
5	<code>filter(function,sequence)</code>	Test each element of the list as true or not on the function passed as an argument thereto and returns a filtered iterator.
6	<code>map(function, iterator)</code>	Returns a list after applying the function passed as an argument to every element of the iterable.
7	<code>lambda()</code>	An anonymous function that may have any number of arguments but one expression, that calculates and returns the result.
8	<code>all()</code>	Returns True if all the items/elements present within the list are true or if the list is empty.
9	<code>any()</code>	Returns True if any of the item(s)/at least one among the items present within the list is/are true. And returns False if the list is empty.
10	<code>sum()</code>	Returns the sum of all the elements present within the list.
11	<code>reduce(func, sequence)</code>	It applies a function passed as an argument to all the elements of the sequence and returns a result based on the function.

Iterating Over a Python List

The list is a type of ordered container that can store heterogeneous and homogenous data. Every element stored in the list has a definite position and a definite count. There are various ways in which

lists can be traversed/iterated. As we advance, we will be going through various practices of iterating through a list

By using for loop: We make use of for loop to traverse from index 0 to end.

```
# Creating List
```

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']
```

```
# Traversing using for loop.
```

```
for character in my_List:
```

```
    print(character, end=' ')
```

OUTPUT:

i n t e r n

By using for loop and range(): We use range() to traverse over the specified indexes or traverse list traditionally. However it is advised to iterate over a list using the previous method, instead of using index in this method.

```
# Initializing a List
```

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']
```

```
size = len(my_List)
```

```
# Traversing using for loop.
```

```
for i in range(size):
```

```
    print(my_List[i], end = ' ')
```

OUTPUT:

i n t e r n

By using while loop:

```
# Initializing a List
```

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']
size = len(my_List)

i = 0

# Traversing using a while loop.
while i < size:
    print(my_List[i], end = ' ')
    i += 1
```

OUTPUT:

i n t e r n

List comprehension: Using List comprehension is one of the best possible ways to iterate over a list. List comprehension makes the program look more elegant and easily understandable.

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']

# iterating using List Comprehension
[print(x, end = ' ') for x in my_List]
```

OUTPUT:

i n t e r n

Using the enumerate(): enumerate() in python allows us to loop through elements as well as the index within the list. This is helpful when we want to traverse only certain index elements in the list based on certain condition(s).

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']

# iterating using enumerate
for i, item in enumerate(my_List):
    if i%2 == 0:
        print(i, item)
```

OUTPUT:

0 i

2 t

4 r

To learn about Data Types in Python, Click Here.

notes

After learning about Lists in Python, here are some key points that will help you:

List is mutable and comes with lots of inbuilt functions.

List is always preferred among all available sequences.

List comprehension is useful in terms of short condition programs.

There are multiple built in list functions.

Overview

Comprehension in python provide a concise and expressive way to create lists, dictionaries, and sets. They allow you to generate these data structures in a single line of code, making your code more readable and efficient.

Types of Comprehensions in Python

List comprehensions in Python

List comprehensions in Python provide a concise way to create lists. They allow you to create a new list by applying an expression to each item in an existing iterable (e.g., a list, tuple, or string), optionally with a condition to filter items.

Syntax:

`new_list = [<expression> for <item> in <iterable> if <condition>]`

`new_list`: new list being created.

`expression`: refers to the expression that would be used to manipulate the elements in the list. It could be a formula or a simple value.

`item`: refers to the original element in the list.

`iterable`: refers to the original sequence being used to create a `new_list`.

`condition`: It is an optional parameter used to reduce the length of the `new_list`, making it shorter than the original sequence.

Now, let's understand this better with some examples.

Example 1:

Considering the example of converting a list of red balls to red. All we do is create a `red_list` from the existing `blue_list` by naming each element as red in the new list.

```
blue_list= ["blue", "blue", "blue", "blue", "blue"]
```

```
print(blue_list)
```

```
red_list = ["red" for list_item in blue_list]
```

```
print(red_list)
```

Output:

```
['blue', 'blue', 'blue', 'blue', 'blue']
```

```
['red', 'red', 'red', 'red', 'red']
```

Example 2:

Considering you have a list of consecutive even numbers and want to create a list of consecutive odd numbers, you can do it by adding 1 to every element of the list. This is achieved by considering the item in `even_list` as var and then adding 1 to it, as shown in the following code:

```
even_list = [x for x in range(0,11,2)]  
print(even_list)  
odd_list = [var+1 for var in even_list]  
print(odd_list)
```

Output:

```
[0, 2, 4, 6, 8, 10]  
[1, 3, 5, 7, 9, 11]
```

Dictionary comprehensions in Python?

Dictionary comprehensions are used to create a fresh dictionary from an existing sequence of dictionaries or lists by manipulating its elements. They have a similar syntax to list comprehensions and set comprehensions but are used to generate dictionaries.

Syntax:

```
new_dict = [<key>:<value> for <item> in <iterable> if <condition>]
```

`new_dict`: new dictionary being created.

`key`: The expression for the dictionary key.

`value`: The expression for the dictionary value.

`item`: The variable representing an element in the original iterable.

`iterable`: The existing sequence you are iterating over.

`condition (optional)`: An optional filtering condition. The item is included in the new dictionary only if this condition is true.

Let's understand this better with some examples.

Example 1:

Consider a list of dictionary objects containing words and their meanings where each dictionary is in the format:

```
{  
    'word': 'insert_word_here',  
    'meaning': 'insert_meaning_here',  
}
```

We aim to create a sequence of dictionaries where each dictionary object is in the format:

```
{  
'insert_word_here': 'insert_meaning here'  
}
```

To do so, we create a new dictionary using dictionary comprehension as follows:

```
word_meaning = [  
    {  
        'word': 'accomplice',  
        'meaning': 'a person who helps another commit a crime.',  
    },  
    {  
        'word': 'brutal',  
        'meaning': 'savagely violent.',  
    },  
]
```

```

{
    'word': 'cacophony',
    'meaning': 'a harsh discordant mixture of sounds.',
},
{
    'word': 'Accomplice',
    'meaning': 'extremely serious or urgent.',
},
]
print(word_meaning)

new_dict = {d['word']:d['meaning'] for d in word_meaning}
print(new_dict)

```

In the above code, d['word'] represents the value held by the key, word and the d['meaning'] represents the value held by the key meaning.

Output:

```

[{'word': 'accomplice', 'meaning': 'a person who helps another commit a crime.'}, {'word': 'brutal',
'meaning': 'savagely violent.'}, {'word': 'cacophony', 'meaning': 'a harsh discordant mixture of sounds.'},
{'word': 'Accomplice', 'meaning': 'extremely serious or urgent.'}]

{'accomplice': 'a person who helps another commit a crime.', 'brutal': 'savagely violent.', 'cacophony': 'a
harsh discordant mixture of sounds.', 'Accomplice': 'extremely serious or urgent.'}

```

Example 2:

Consider the following example of 1 list, one containing the months of a year and another consisting of natural numbers up to 12. We aim to create a dictionary where the key is the number of the month, and the value is the month's name. A dictionary using dictionary comprehension for the given purpose would be created as follows:

```
numbers = [x for x in range(1,13,1)]  
month_of_y = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']  
print(numbers)  
print(month_of_y)  
year_dict = {key:value for (key, value) in zip(numbers, month_of_y)}  
print(year_dict)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]  
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']  
{1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr', 5: 'May', 6: 'Jun', 7: 'Jul', 8: 'Aug', 9: 'Sep', 10: 'Oct', 11: 'Nov', 12: 'Dec'}
```

Set comprehensions in Python

In Python, set comprehensions are used to create a new set from an existing sequence (set) by manipulating its elements. They have a similar syntax to list comprehensions, but they generate sets instead.

Syntax:

```
new_set = [<expression> for <item> in <iterable> if <condition>]
```

new_set: new set being created.

expression: The operation or value to be included in the new set.

item: The variable representing an element in the original iterable.

iterable: The existing sequence you are iterating over.

condition (optional): An optional filtering condition. The item is included in the new set only if this condition is true.

Let's understand this better with some examples.

Example 1:

Considering the given example of converting a set of red balls to red. All we do is create a red_set from the existing blue_set by naming each element as red in the new set as follows:

```
red_set = {"red" for set_item in blue_set}
```

The entire code for this example is given below:

```
blue_set= {"blue", "blue", "blue", "blue", "blue"}  
print(blue_set)  
red_set = {"red" for set_item in blue_set}  
print(red_set)
```

Output:

```
{'blue'}  
{'red'}
```

Example 2:

Consider you have a list of consecutive even numbers and want to create a set of consecutive odd numbers. You can do it by adding 1 to every element of the set. This is done by considering each item in the existing set even_set as var and then adding 1 to var, as shown in the code below:

```
even_set = {x for x in range(0,11,2)}  
print(even_set)  
odd_set = {var+1 for var in even_set}  
print(odd_set)
```

Output:

```
{0, 2, 4, 6, 8, 10}
```

```
{1, 3, 5, 7, 9, 11}
```

Generator comprehensions in Python

This is one of the most rarely used and least known comprehensions in Python. In Python, generator comprehensions are used to create a new generator from an existing list by manipulating its elements. A generator in Python is a function that returns an iterator that holds a sequence of values instead of a single value. Generators are usually memory efficient as they allocate memory on the generation of items rather than at the beginning.

Syntax:

```
new_gen = [<expression> for <item> in <iterable> if <condition>]
```

`new_gen`: new generator being created.

`expression`: The operation or value to be included in the new set.

`item`: The variable representing an element in the original iterable.

`iterable`: The existing sequence you are iterating over.

`condition (optional)`: An optional filtering condition. The item is included in the new set only if this condition is true.

Now, let's understand this better with an example.

Consider you have been given a list of the first 6 even numbers, and you want to generate a list consisting of the first 6 odd numbers from it. You would simply add a 1 to the numbers given in the existing list. The same can be coded as follows:

```
even_list = [x for x in range(0,11,2)]
```

```
print(even_list)
```

```
odd_set = (var+1 for var in even_list)
for var1 in odd_set:
    print(var1, end=" ")
```

Output:

[0, 2, 4, 6, 8, 10]

1 3 5 7 9 11

Notes:

Comprehension in Python are used to create new sequences from existing ones, and the type of the new and original sequence might not be the same.

Comprehension in Python are classified into four types based on the sequence generated. These are list comprehensions, dictionary comprehensions, set comprehensions, and generator comprehensions.

Comprehension in Python can be used to make the code more readable and also make the code run faster in most cases since Python first allocates memory for the list and then adds elements.

Comprehension in Python are more often used when you want to sophisticatedly filter lists, use conditional logic to create them, or map elements.